

OWUS

 [maoyab/OWUS](#)

 english

 gemini-2.5-flash

 7bb8fe8

 Dec 9, 2025

Chapters

Overview of OWUS

Chapter 1: Ecohydrological Data Preparation

Chapter 2: Stochastic Soil Water Balance Model (SSWM)

Chapter 3: Water Use Strategy Framework

Chapter 4: Model Parameter Inference (MCMC)

Chapter 5: Scientific Visualization

Show Raw Markdown

Chapter 5: Scientific Visualization

Welcome back! In [Chapter 1: Ecohydrological Data Preparation](#), we learned to prepare our raw environmental data. Then, in [Chapter 2: Stochastic Soil Water Balance Model \(SSWM\)](#), we used that data to simulate a plant's long-term water balance. [Chapter 3: Water Use Strategy Framework](#) introduced how we compare different plant "game plans," and [Chapter 4: Model Parameter Inference \(MCMC\)](#) showed us how to estimate the specific parameters that define these strategies from observed data.

Now, we have all these amazing results: clean data, simulated soil moisture patterns, transpiration rates, water use efficiency values, and inferred plant parameters. But what do these numbers *mean*? How do we effectively communicate our complex scientific findings to others, or even understand them ourselves? This chapter introduces **Scientific Visualization**,



the crucial final step in our **OWUS** journey.

What Problem Does Scientific Visualization Solve? Making Sense of Complex Data!

Imagine you're a detective who has just solved a very complex case. You have stacks of evidence: fingerprints, witness statements, timestamps, financial records – a mountain of raw information. Simply presenting a list of these facts won't convince anyone. Instead, you need to piece together a compelling visual story: maps showing movements, timelines of events, diagrams illustrating connections.

The central problem: In scientific research, raw numerical data, model outputs, and inferred parameters can be incredibly complex and difficult to interpret. Trying to understand pages of numbers or a long list of parameter values is not only tedious but also prone to misinterpretation. We need a way to:

1. **Understand:** Quickly grasp patterns, trends, and relationships within our data.
2. **Verify:** Visually check if our models are performing as expected by comparing simulations with observations.
3. **Communicate:** Effectively convey our findings to other scientists, stakeholders, or the public in a clear, impactful way.

This is where **Scientific Visualization** comes in. It's like having a skilled graphic designer for your scientific data, transforming raw numbers into clear, informative, and visually appealing charts and illustrations.

A concrete example: For our 'AT-Neu' grassland site, after all the hard work in previous chapters, we want to visually answer questions like:

- How does the **SSWM**'s predicted soil moisture distribution compare to the actual observed distribution?
- How do the "data-driven" and "optimality-based" water use strategies (from [Chapter 3: Water Use Strategy Framework](#)) differ in terms of the soil moisture they predict or the water use efficiency they achieve?
- Are there clear relationships between different environmental factors and a plant's water use strategy?

This chapter will guide you through how **OWUS** helps you generate these visual stories to effectively communicate your complex ecohydrological findings.

Key Concepts: Understanding Visual Tools

Scientific visualization uses various types of plots and figures to represent different aspects of your data. Let's look at some common ones `OWUS` employs:

1. Probability Distributions (Histograms / PDFs)

Often, our models don't just give a single value, but a *range* of likely values (like the soil moisture probability distribution, p_0 , from [Chapter 2: Stochastic Soil Water Balance Model \(SSWM\)](#), or parameter uncertainty from [Chapter 4: Model Parameter Inference \(MCMC\)](#)).

- **What they show:** How frequently different values occur within a dataset.
- **Example:** A histogram of soil moisture might show that the soil is usually moderately wet, but occasionally very dry.

2. Time Series Plots

Many ecohydrological variables change over time (e.g., daily rainfall, temperature, Leaf Area Index (LAI) from [Chapter 1: Ecohydrological Data Preparation](#)).

- **What they show:** How a variable evolves or fluctuates over a period.
- **Example:** Plotting observed rainfall and simulated soil moisture over a year helps to see how soil moisture responds to rain events.

3. Scatter Plots

These plots are excellent for revealing relationships between two different variables.

- **What they show:** If two variables tend to increase or decrease together (positive/negative correlation), or if there's no clear pattern.
- **Example:** Plotting modeled transpiration against observed transpiration to assess model accuracy, or plotting aridity against water use efficiency to see how plant strategy changes with climate.

4. Functional Plots (e.g., beta(s) curves)

Some of our models describe how a process changes depending on a key input. For example, how transpiration changes as soil moisture varies.

- **What they show:** The shape of a relationship or function, often showing thresholds or non-linear responses.
 - **Example:** The $\beta(s)$ curve (fraction of transpiration to potential demand as a function of soil saturation) is a central concept in `OWUS`, illustrating a plant's water use strategy.
-

How to Use OWUS for Visualization: Bringing Data to Life

OWUS uses the popular Python library `matplotlib` (and sometimes `seaborn` for aesthetics) to create all its plots. Instead of writing `matplotlib` code from scratch for every figure, OWUS provides a set of specialized functions in the `create_figs.py` script. These functions are designed to load results from OWUS workflows (like the inferred parameters from [Chapter 4: Model Parameter Inference \(MCMC\)](#)) and automatically generate the relevant scientific visualizations.

Let's look at a common visualization task: comparing how well our model's predicted soil moisture distributions (for both data-driven and optimality-based strategies) match the observed soil moisture. The `plot_sm_gof` function in `create_figs.py` is perfect for this.

First, you'd typically run your OWUS workflow to generate results, which include saved data frames or pickle files of parameters and model outputs. Then, you'd load these results and call the plotting function:

```
1 import pandas as pd
2 import numpy as np # Used by create_figs internally
3 import matplotlib.pyplot as plt # Used by create_figs internally
4 from create_figs import plot_sm_gof # Our visualization function
5
6 # 1. Load the combined results from data-driven and optimality-based inferences
7 # These CSV files would be generated after running the MCMC (Chapter 4) for multiple
8 res_opt = pd.read_csv('DATA/WUS/WUS_output/combined/results_opt__1.csv')
9 res_dd = pd.read_csv('DATA/WUS/WUS_output/combined/results_bf__1.csv')
10
11 # 2. Call the visualization function to generate the plot
12 print("Generating Soil Moisture Goodness-of-Fit Plot...")
13 plot_sm_gof(res_opt, res_dd, figname='at_neu_sm_comparison')
14 print("Plot generation complete. Check 'PROJECTS/optimal_water_use_strategies/Figures'")
15
16 # The plot will typically be saved as a PDF or PNG in a designated output folder,
17 # for example: PROJECTS/optimal_water_use_strategies/Figures/at_neu_sm_comparison.pdf
18
```

What happens here?

- We load two `pandas DataFrames`: `res_opt` (results from optimality-based parameter inference) and `res_dd` (results from data-driven inference). These would contain the inferred

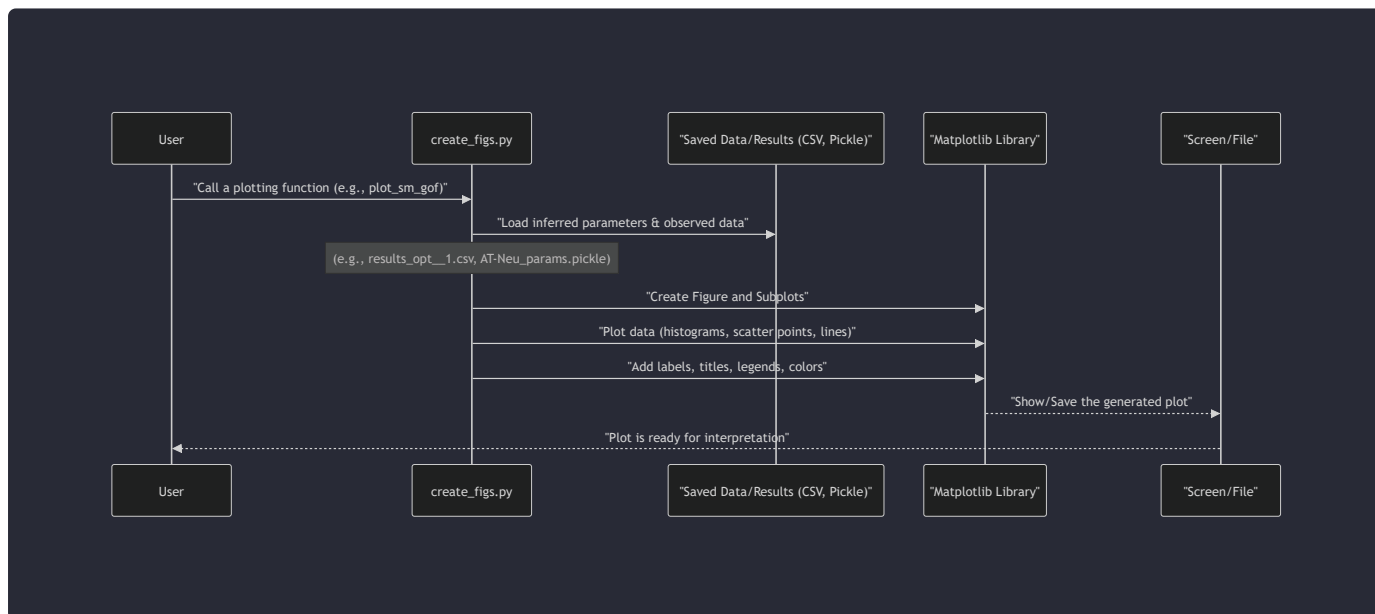
parameters and performance metrics for multiple sites, including our 'AT-Neu' example.

- The `plot_sm_gof` function then takes these results. Internally, it loads the *observed* soil moisture data for each site (from the `DATA/WUS/WUS_output/opt/files/_1/{site}.pickle` files saved during Ecohydrological Data Preparation).
- It calculates goodness-of-fit metrics (like Nash-Sutcliffe Efficiency, NSE, and Pearson correlation, ρ) by comparing the *simulated* soil moisture distributions (derived from the inferred parameters via the Stochastic Soil Water Balance Model (SSWM)) with the *observed* soil moisture.
- Finally, it uses `matplotlib` to create and save two subplots: a histogram comparing the distribution of NSE values for both strategies, and a scatter plot showing modeled vs. observed average soil moisture. This visually summarizes the performance of each strategy across all sites.

Under the Hood: The Visualization Workflow

Scientific visualization in `OWUS` isn't a separate computational model, but rather a **structured process** that applies plotting tools to the outputs generated by earlier chapters. It's about taking the numerical results and presenting them graphically.

Here's a simplified conceptual workflow:



Dive Deeper into Code (`create_figs.py`)

Let's take a closer look at the `plot_sm_gof` function in `create_figs.py` to understand how it uses `matplotlib`. This function focuses on comparing the simulated soil moisture distribution ($p(s)$) with observations.

```

1 # From create_figs.py (simplified for clarity)
2 # ... (imports and helper functions like cal_nse, pearsonr, LinearRegression) ...
3
4 def get_sm_gof(res_opt, res_dd):
5     # This function is called by plot_sm_gof to calculate metrics.
6     # It loops through each site, loads observed soil moisture (s_obs)
7     # and the specific parameters for data-driven (dd) and optimality-based (opt)
8     # strategies. It then uses the SSWM model (SM_C_H) to generate simulated
9     # soil moisture distributions (p0) for each strategy.
10    # It converts p0 to cumulative distributions (CDF) and resamples to
11    # calculate NSE (Nash-Sutcliffe Efficiency) for p(s) and mean soil moisture
12    # correlation (rho) for both strategies against observations.
13    # ... (detailed calculations and SSWM calls omitted for brevity) ...
14    # Returns lists of NSEs and mean soil moistures for dd, opt, and observed.
15    return all_nse_dd, all_nse_opt, all_smean_opt, all_smean_dd, all_smean_obs
16
17
18 def plot_sm_gof(res_opt, res_dd, figname='sm_gof'):
19     # Call the helper function to calculate all the performance metrics
20     all_nse_dd, all_nse_opt, all_smean_opt, all_smean_dd, all_smean_obs = get_sm_gof(
21
22     # 1. Create the main figure and two subplots (arranged in 2 rows, 1 column)
23     fig = plt.figure(figsize=(3, 6)) # Define the overall size of the figure
24     ax_h = fig.add_subplot(2, 1, 1) # Top subplot for the histogram
25     ax_s = fig.add_subplot(2, 1, 2) # Bottom subplot for the scatter plot
26
27     cmap = plt.cm.get_cmap('viridis') # Choose a color map for visual consistency
28
29     # 2. Plot 1: Histogram of NSE values (Goodness-of-Fit)
30     ax_h.hist(all_nse_dd, bins=np.linspace(0, 1, 6), color=cmap(1), alpha=1, label='Data-driven')
31     ax_h.hist(all_nse_opt, bins=np.linspace(0, 1, 6), color=cmap(0.5), alpha=0.75, label='Optimality-based')
32     ax_h.set_xlabel('NSE between empirical \n and theoretical $p(s)$', fontsize=12)
33     ax_h.set_ylabel('Number of sites', fontsize=12)
34     ax_h.set_xlim([0,1])
35     ax_h.set_title('(a)') # Subplot title
36     ax_h.legend(frameon=False) # Display the legend without a box
37
38     # 3. Plot 2: Scatter plot of long-term average soil moisture (Modeled vs. Observed)
39     ax_s.scatter(all_smean_dd, all_smean_obs, color=cmap(1), alpha=1) # Data-driven
40     ax_s.scatter(all_smean_opt, all_smean_obs, color=cmap(0.5), alpha=0.75) # Optimality-based
41
42     # Calculate and plot linear regression lines through the origin for each strategy

```

```

43     reg_dd = LinearRegression(fit_intercept=False).fit(np.array(all_smean_dd).resid
44     reg_opt = LinearRegression(fit_intercept=False).fit(np.array(all_smean_opt).resid
45     ax_s.plot([0, 1], [0, 1*reg_dd.coef_[0]], linestyle="--", color=cmap(1), label=
46     ax_s.plot([0, 1], [0, 1*reg_opt.coef_[0]], linestyle="--", color=cmap(0.5), lab
47
48     ax_s.plot([0,1], [0,1], linestyle="--", color='silver') # Plot the 1:1 line fo
49     ax_s.set_xlabel(r'$\langle s \rangle$ (modeled)', fontsize=12)
50     ax_s.set_ylabel(r'$\langle s \rangle$ (observed)', fontsize=12)
51     ax_s.set_title('(b)') # Subplot title
52     ax_s.set_xlim([0.1, 0.85]) # Set x-axis limits
53     ax_s.set_ylim([0.1, 0.85]) # Set y-axis limits
54
55     plt.tight_layout() # Adjust layout to prevent labels/titles from overlapping
56     # 4. Save the generated figure to a file
57     plt.savefig(f'../PROJECTS/optimal_water_use_strategies/Figures/{figname}.pdf')
58     # plt.show() # Uncomment this line to display the plot immediately on your screen
59

```

Key Takeaways from the Code:

- The `plot_sm_gof` function orchestrates the creation of a multi-panel figure, bringing together complex data from various steps of the `OWUS` workflow.
- `fig.add_subplot(rows, columns, panel_number)` is used to arrange multiple plots (e.g., histograms and scatter plots) within one figure.
- Standard `matplotlib` methods like `hist()` for histograms and `scatter()` for scatter plots are used to visualize the data.
- `set_xlabel()`, `set_ylabel()`, `set_title()`, and `legend()` are used to add context and clarity to the plots.
- `plt.savefig()` is crucial for saving the high-quality output figure to a file (e.g., PDF), making it ready for presentations or publications.

This example illustrates how `OWUS` provides pre-built, high-quality visualization functions, allowing users to focus on interpreting results rather than spending time on writing detailed plotting code.

Conclusion

In this chapter, we've explored the power of **Scientific Visualization** in the `OWUS` project. We learned that transforming complex numerical data into clear visual representations is essential for

understanding, verifying, and communicating our ecohydrological findings. We saw how `OWUS` leverages `matplotlib` within specialized functions (like `plot_sm_gof` in `create_figs.py`) to generate insightful figures that compare model predictions with observations and reveal the implications of different plant water use strategies.

By making complex data accessible and understandable, scientific visualization completes the loop of our research process, turning raw information into actionable scientific knowledge.

Congratulations! You've now completed the core tutorial for the `OWUS` project. You've learned how to prepare data, simulate soil water balance, understand water use strategies, infer model parameters, and visualize your results. This comprehensive understanding will empower you to apply `OWUS` to new research questions and contribute to our understanding of plant water dynamics in a changing world. Happy visualizing!

Generated by [AI Codebase Knowledge Builder](#). **References:** [1], [2], [3], [4]

© 2025 Codebase to Tutorial. All rights reserved.

[Terms of Service](#) [Privacy Policy](#)